

Spring Core Fundamentals

Notes based on the Spring Core fundamentals lecture.



Table of Contents

1. [Introduction](#)
 2. [Dependency Injection](#)
 3. [Problem with Manual Dependency Creation](#)
 4. [Inversion of Control](#)
 5. [IoC Container](#)
 6. [Spring Context Dependency](#)
 7. [Beans](#)
 8. [Spring Configuration Approaches](#)
 9. [Java Reflection API](#)
 10. [Component Annotation](#)
 11. [ApplicationContext](#)
 12. [Configuration Class](#)
 13. [Component Scanning](#)
 14. [Complete Flow](#)
 15. [Key Takeaways](#)
-

1. Introduction

Spring Framework is built around **Spring Core**.

The important Spring Core concepts discussed are:

- Dependency Injection (DI)
- Inversion of Control (IoC)
- IoC Container
- Beans
- ApplicationContext
- Component Scanning
- Annotations
- Java Reflection API

The main objective is to understand how Spring creates and manages objects instead of the developer manually creating and connecting them.

2. Dependency Injection

What is Dependency Injection?

Dependency Injection means:

A class receives its required dependency from outside instead of creating the dependency itself.

Example

Suppose we have:

```
class PaymentService {  
  
    public void pay() {  
        System.out.println("Payment Done");  
    }  
}
```

And:

```
class OrderService {  
  
    private PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
  
    public void placeOrder() {  
        paymentService.pay();  
        System.out.println("Order Placed");  
    }  
}
```

Here:

```
OrderService  
  |  
  ↓  
PaymentService
```

`OrderService` depends on `PaymentService`.

Therefore:

`PaymentService` is a dependency of `OrderService`.

3. Problem with Manual Dependency Creation

One way to create the dependency is:

```
class OrderService {  
  
    private PaymentService paymentService;  
  
    public OrderService() {  
        paymentService = new PaymentService();  
    }  
}
```

Problem

`OrderService` is now responsible for creating `PaymentService`.

This creates tight coupling and violates the idea of keeping responsibilities separate.

The lecture relates this to the **Single Responsibility Principle**.

`OrderService` should primarily handle orders, not create and manage payment-service objects.

4. Inversion of Control

What is IoC?

IoC = Inversion of Control

Normally, the developer controls:

```
Object Creation  
    ↓  
Dependency Creation
```

↓
Dependency Wiring
↓
Object Management

With Spring:

Spring Container
↓
Creates Objects
↓
Wires Dependencies
↓
Manages Objects

So the control of object creation and management is transferred from the developer to the Spring Framework.

This is called:

Inversion of Control

5. IoC Container

Spring provides an **IoC Container**.

The IoC Container is responsible for managing application objects.

It handles things such as:

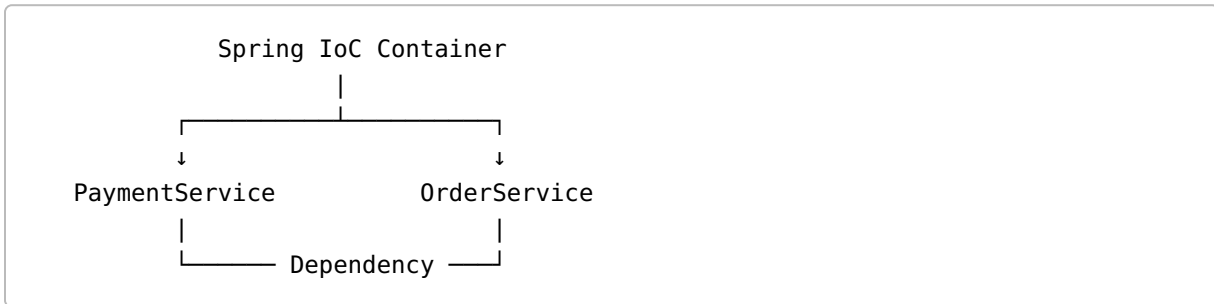
- Creating objects
- Managing objects
- Wiring dependencies
- Injecting dependencies
- Managing the lifecycle of objects

Without Spring

```
PaymentService paymentService = new PaymentService();  
  
OrderService orderService =  
    new OrderService(paymentService);
```

The developer manually performs the wiring.

With Spring



Spring performs the required object creation and dependency wiring.

6. Spring Context Dependency

For basic Spring Core functionality, the lecture uses:

```
spring-context
```

Why?

Because the application needs Spring's core container functionality.

The lecture specifically distinguishes this from:

Spring Boot

Spring Boot primarily provides easier/automatic configuration.

Spring MVC

Spring MVC is used for web applications.

But the example in the lecture is a simple console-based application.

Therefore:

Basic Spring Core Application

↓

spring-context

7. Beans

What is a Bean?

In Spring, objects managed by the Spring IoC Container are called:

Beans

For example:

```
PaymentService  
OrderService
```

If Spring creates and manages their objects, they are Spring Beans.

Important Relationship

```
Bean = Object managed by Spring
```

But:

```
Every Bean is an Object  
BUT  
Every Object is NOT necessarily a Bean
```

An object becomes a Spring Bean when it is managed by the Spring container.

8. Spring Configuration Approaches

The lecture introduces two ways of configuring Spring.

8.1 Annotation-Based Configuration

Uses annotations such as:

```
@Component  
@Configuration  
@ComponentScan
```

This is the modern approach discussed first in the lecture.

8.2 XML-Based Configuration

Spring can also be configured using XML.

This approach is older and is more commonly encountered in legacy Spring applications.

Comparison

Configuration	Usage
Annotation-based	Common in modern applications
XML-based	Mostly seen in older/legacy applications

The lecture focuses first on **annotation-based configuration**.

9. Java Reflection API

Spring needs information about classes to create and manage their objects.

This is where the **Java Reflection API** becomes important.

9.1 What is Reflection?

Reflection allows Java to inspect information about a class at runtime.

For example:

```
class Student {
```

```
private String name;
private int age;

public Student() {
}

public void getAttendance() {
}

public void printStudent() {
}
}
```

We can obtain the class metadata using:

```
Class<Student> c1 = Student.class;
```

9.2 What is Class?

Java has a special class called:

Class

It represents metadata about a class.

Example:

```
Class<Student> c1 = Student.class;
```

`c1` is **not** a `Student` object.

Instead, it represents information about the `Student` class.

9.3 What Metadata Can Be Available?

Class metadata can contain information such as:

Class Name
Fields

Field Types
Constructors
Methods
Access Modifiers
Annotations

Conceptually:

```
Student.class
  |
  ↓
Class Metadata
  |
  ├── Class Name
  ├── Fields
  ├── Constructors
  ├── Methods
  └── Annotations
```

Spring uses this type of class metadata to understand classes and work with them at runtime.

10. @Component

Now Spring needs to know:

Which classes should Spring manage?

For this, we use:

@Component

Example:

```
@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment Done");
    }

}
```

And:

```
@Component
public class OrderService {

    public void placeOrder() {
        System.out.println("Order Placed");
    }
}
```

Purpose of @Component

@Component acts as a marker telling Spring:

"Manage objects of this class."

Therefore:

```
@Component
  ↓
Spring detects class
  ↓
Spring creates its object
  ↓
Object becomes a Bean
  ↓
Spring manages it
```

11. ApplicationContext

The Spring IoC Container is represented through:

ApplicationContext

ApplicationContext is an interface.

Therefore, we cannot directly do:

```
new ApplicationContext();
```

because it is an interface.

Annotation-Based Implementation

For annotation-based configuration, the lecture uses:

```
AnnotationConfigApplicationContext
```

Example:

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(...);
```

This means:

Start the Spring IoC Container using annotation-based configuration.

12. Configuration Class

Spring needs configuration information telling it:

- Which classes should be managed?
- Where should Spring search?
- Which configuration rules should be used?

For this, we create a configuration class.

Example:

```
@Configuration  
@ComponentScan("in.coderarmy")  
public class AppConfig {  
  
}
```

13. @Configuration

The annotation:

```
@Configuration
```

tells Spring:

This class contains Spring configuration.

Example:

```
@Configuration
public class AppConfig {

}
```

This class becomes the source of configuration information for the Spring container.

14. @ComponentScan

Spring needs to know where to search for components.

For that, we use:

```
@ComponentScan("in.coderarmy")
```

Example:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {

}
```

How @ComponentScan Works

Suppose our project structure is:

```
in.coderarmy
|
├─ Main
├─ AppConfig
├─ OrderService
└─ PaymentService
```

And:

```
@Component
class OrderService {
}
```

```
@Component
class PaymentService {
}
```

Then:

```
@ComponentScan("in.coderarmy")
```

tells Spring:

```
Search in.coderarmy
    ↓
Find @Component classes
    ↓
OrderService
PaymentService
    ↓
Create their objects
    ↓
Manage them as Beans
```

15. Starting the Spring Container

After creating the configuration class:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {

}
```

we pass its class information to:

```
AnnotationConfigApplicationContext
```

Example:

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

What does `AppConfig.class` mean?

```
AppConfig.class
```

provides Spring with metadata about the configuration class.

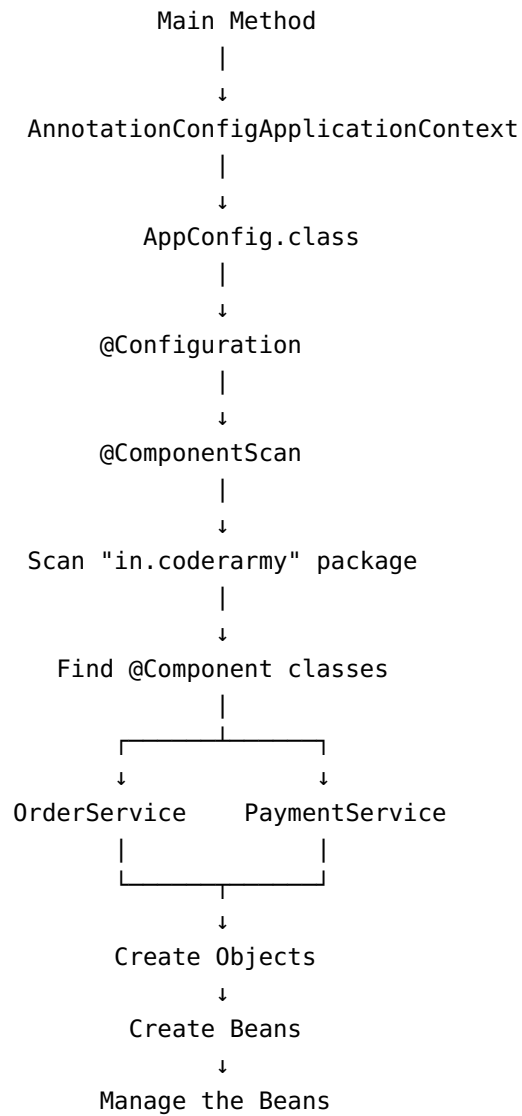
Spring can inspect:

```
@Configuration
@ComponentScan("in.coderarmy")
```

and understand the configuration rules.

16. Complete Spring Startup Flow

The complete flow discussed so far is:



17. Dependency Wiring

Consider:

```
@Component
public class OrderService {

    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

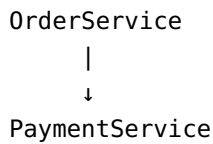
```

    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order Placed");
    }
}

```

Here:



`OrderService` requires `PaymentService`.

Spring's container can manage this dependency instead of the developer manually creating and passing the object.

18. Important Terminology

Term	Meaning
Dependency	Object required by another object
Dependency Injection	Providing dependency from outside
IoC	Transfer of object-management control to Spring
IoC Container	Container responsible for creating/managing objects
Bean	Object managed by Spring
ApplicationContext	Spring interface representing the application container
<code>@Component</code>	Marks a class for Spring management
<code>@Configuration</code>	Marks a configuration class
<code>@ComponentScan</code>	Tells Spring where to search for components
Reflection	Runtime inspection of class metadata

19. Dependency Injection vs IoC

These concepts are related but not identical.

Dependency Injection

Focuses on:

How dependencies are provided

Example:

```
OrderService(PaymentService paymentService)
```

Inversion of Control

Focuses on:

Who controls object creation and management?

Without Spring:

```
Developer
  ↓
Creates objects
  ↓
Wires dependencies
```

With Spring:

```
Spring Container
  ↓
Creates objects
  ↓
Wires dependencies
  ↓
Manages beans
```

20. Important Code Structure

A basic annotation-based Spring Core project from the lecture can be visualized as:

```
src
├── main
│   └── java
│       └── in.coderarmy
│           ├── Main.java
│           ├── AppConfig.java
│           ├── OrderService.java
│           └── PaymentService.java
```

PaymentService

```
@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment Done");
    }
}
```

OrderService

```
@Component
public class OrderService {

    public void placeOrder() {
        System.out.println("Order Placed");
    }
}
```

AppConfig

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {

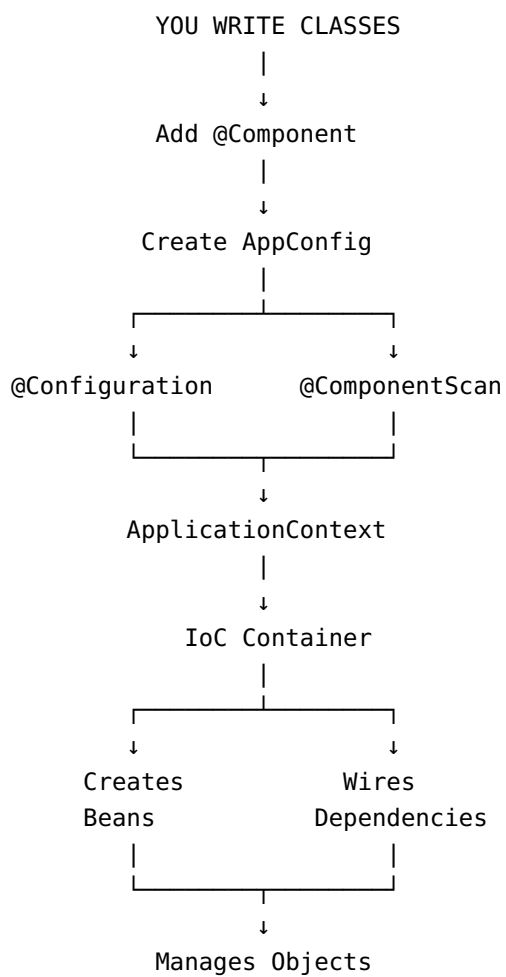
}
```

Main

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

21. Mental Model

Remember Spring Core using this simple model:



22. Quick Revision

Dependency Injection

Dependency is provided externally.

Inversion of Control

Spring takes control of object creation and management.

IoC Container

Creates + wires + manages Beans.

Bean

Object managed by Spring.

@Component

Marks a class to be managed by Spring.

@Configuration

Marks a configuration class.

@ComponentScan

Tells Spring where to search for components.

ApplicationContext

Represents the Spring application container.

Reflection

Provides runtime access to class metadata.

23. Interview Questions

Q1. What is Dependency Injection?

Dependency Injection is a technique where a class receives its required dependencies from outside instead of creating them itself.

Q2. What is Inversion of Control?

IoC means transferring the responsibility of object creation and management from the developer to the Spring Framework.

Q3. What is an IoC Container?

It is the Spring container responsible for creating, configuring, wiring, and managing Spring Beans.

Q4. What is a Spring Bean?

A Spring Bean is an object that is created and managed by the Spring IoC Container.

Q5. What does `@Component` do?

`@Component` marks a class so that Spring can detect it during component scanning and manage its object as a Bean.

Q6. What is `ApplicationContext`?

`ApplicationContext` is a Spring interface that represents the application IoC container.

Q7. What is `AnnotationConfigApplicationContext`?

It is an implementation used to create a Spring application context using annotation-based configuration.

Q8. What is `@ComponentScan`?

It tells Spring which package(s) should be scanned for Spring components.

Q9. Why is Reflection important in Spring?

Spring needs class metadata to understand classes and manage their objects at runtime. Java Reflection provides access to this metadata.

Q10. What is the difference between an Object and a Bean?

Object → Any Java object

Bean → Object managed by Spring

Therefore:

Every Bean is an Object

Every Object is NOT necessarily a Bean

24. Final Summary

The core idea of the lecture is:

```
Dependency Injection
    ↓
Inversion of Control
    ↓
IoC Container
    ↓
ApplicationContext
    ↓
Beans
```

```
↓  
@Component  
↓  
@Configuration  
↓  
@ComponentScan  
↓  
Spring creates & manages objects  
↓  
Spring handles dependency wiring
```

The most important thing to remember is:

Instead of manually creating and connecting objects, we tell Spring which objects it should manage, and the Spring IoC Container takes responsibility for creating, wiring, and managing those objects.